

1.

用户级线程：采用第三方库来创建的线程。管理线程的所有工作都由应用程序来完成，内核意识不到线程的存在。

内核级线程：操作系统创建的线程。线程的管理工作由内核完成，应用级没有线程管理代码，只有一个内核线程设施的应用编程接口。用户感受不到它们，操作系统可以。

区别：用户级线程由应用程序通过线程库来创建，而内核级程序由操作系统创建。

用户级线程由应用程序来管理，而内核级线程是由操作系统来管理。

用户级线程内核感受不到它们的存在，内核级线程用户感受不到它们的存在。

当线程不需要系统调用和进入内核；要求控制管理简单自主（或者是需要频繁地切换销毁线程时）；要求线程管理开销小；要求线程在不同的操作系统上都能运行；较少占用内核资源时用户级线程更加占优势。

当在多核 cpu 上运行（需要充分利用多核 CPU 优势时），对线程调度优化和 cpu 利用率有较高要求时内核级线程比用户级线程更有优势。

2.

BC 多线程程序中堆内存和全局变量是共享的。

3.

会创建 6 个进程（包括主进程是 6 个，fork 创建了 5 个）2 个线程。（线程数目这里是通过函数显示创建的线程数目）

4.

LINE C 输出 5，LINE P 输出 0。因为 value 是全局变量存放在数据段中，创建子进程时会复制父进程的数据段，所以父子进程的数据段是独立的，子进程创建线程时，因为同一个进程的线程之间共用数据段，所以 value 被共用，所以线程中对 value 的修改就是修改的子进程中的，而父进程与子进程独立，父进程中的数据段没有被修改所以保持原值为 0。

5.

第一种情况下，有些处理器会处于空闲状态，cpu 利用率不高，性能不太好。

第二种情况下，有可能所有处理器将同时使用，但遇到页错误等情况时相应的处理器会闲置。

第三种情况下，所有处理器都会使用，当有一个线程阻塞，会有其他线程调入，保证多处理器系统一直在运行，利用率很高。

选做：思路是编写一个类去传递参数和承接返回的值，做法见代码，我在主程序和线程处理程序中都打印了结果。

结果如下：

```
user@user-virtual-machine: ~/CPP
user@user-virtual-machine:~/CPP$ LS
LS: 未找到命令
user@user-virtual-machine:~/CPP$ ls
hello.cpp  hello.o  homework4.cpp  homework4.o
user@user-virtual-machine:~/CPP$ ./homework4.o
没有输入参数，退出程序! user@user-virtual-machine:~/CPP$
user@user-virtual-machine:~/CPP$ ./homework4.o 22 4 666 44 34 23 123 321
average : 154.62
average:154.62
max : 666
max:666
min : 4
min:4
user@user-virtual-machine:~/CPP$
```